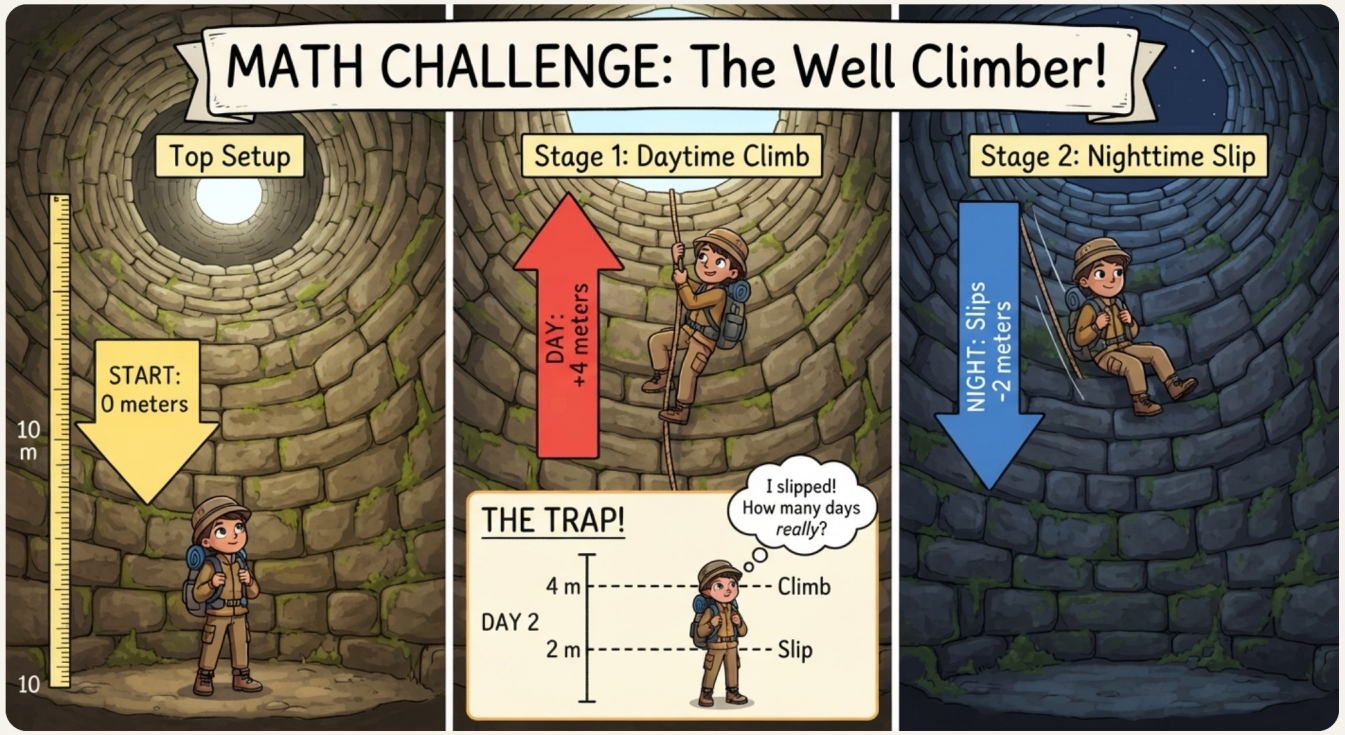
CSE-1003 • MODULE 3 • CLASS 3

# The function that calls itself

Recursion: a problem that contains a smaller copy of itself. And when a plain loop is the better tool.

WARM-UP PUZZLE • THINK FIRST, THEN ANSWER

# Puzzle 1 • The Well Climber




A climber is at the bottom of a **10 m** deep well.
Every **day** he climbs up **4 m.**

Every **night** he slips down **2 m.**

**On which day does he reach the top?**

| Day 1 | Day 2 | Day 3 | Day 4 |
|---|---|---|---|
| 0 → 4 → 2 m | 2 → 6 → 4 m | 4 → 8 → 6 m | 6 → 10 m. Out! |

WARM-UP PUZZLE • THINK FIRST, THEN ANSWER

# Puzzle 2 • The Endless Gift Card

**Starting balance**

$50

in a digital wallet

**Buy 1 game: costs $10**

**Cash-back: +$4 instantly**

**The same idea, as a recursion**

```
games(b) = 1 + games(b - 6)
stop when b < 10: games = 0
```

You start with **$50**. Every game costs **$10** up front, but instantly gives you **$4** back.

**How many TOTAL games can you buy before you run out of spending money?**

**Each game: −$10 +$4 = −$6 in the wallet. You need at least $10 to buy a game.**

| Start | Game 1 | Game 2 | Game 3 | Game 4 | Game 5 | Game 6 | Game 7 |
|---|---|---|---|---|---|---|---|
| $50 | $44 | $38 | $32 | $26 | $20 | $14 | $8 |

WARM-UP PUZZLE • THINK FIRST, THEN ANSWER

# Puzzle 3 • Handshakes at a party




Friends arrive one by one. The **+n** under each person is how many hands they shake when they arrive: only people already there.

**The same idea, as a recursion**

```
handshakes(n) = handshakes(n - 1) + (n - 1)
handshakes(1) = 0
```

**5 friends** meet at a party. Everyone shakes hands with **everyone else**, exactly once.

**How many handshakes happen in total?**

**Think for 1 minute. Write your answer.**

**10 handshakes: 0 + 1 + 2 + 3 + 4**

FROM PUZZLES TO TODAY'S CLASS

# Three puzzles, one idea

### The well

each day starts from where the last day ended

```
climb(h) → climb(h + 2)
```

**stops when the climber reaches 10 m**

### The gift card

each game starts with a smaller balance

```
games(b) = 1 + games(b - 6)
```

**stops when b < 10**

### Handshakes

each new friend adds to a smaller party

```
hs(n) = hs(n - 1) + (n - 1)
```

**stops when n = 1**

**That is recursion**

The same question again, with a **smaller or updated input**, until a **stopping rule**. Today we make a function do exactly this: **a function that calls itself**.

TODAY'S SYLLABUS TOPICS

# Two topics, and a few surprises

1

## Recursive functions

A function that calls itself on a smaller problem. The base case, the recursive case, and the stack growing and shrinking.

2

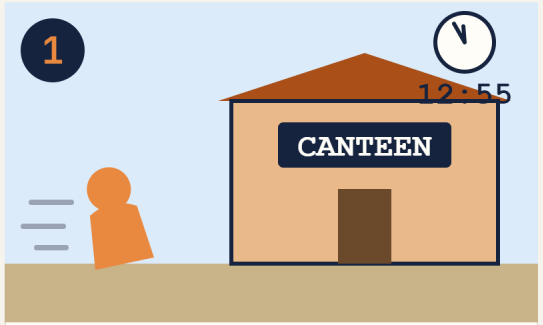
## Recursion vs iteration

The same problem as a loop or as a recursion. Which uses more memory, which is faster, and when to choose which.

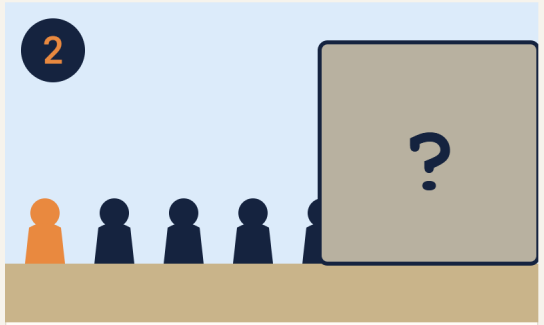
**Along the way** the queue question, stack overflow, climbing stairs, Tower of Hanoi, folders inside folders.

BEFORE WE START • A DAILY-LIFE STORY

# Riya is late for lunch




12:55. The lunch bell rings and Riya runs to the canteen.




The queue goes all the way round the corner.




The break is only 15 minutes. Is lunch worth the wait?




She needs a number: how many people are ahead of me? But she cannot see past the person in front.

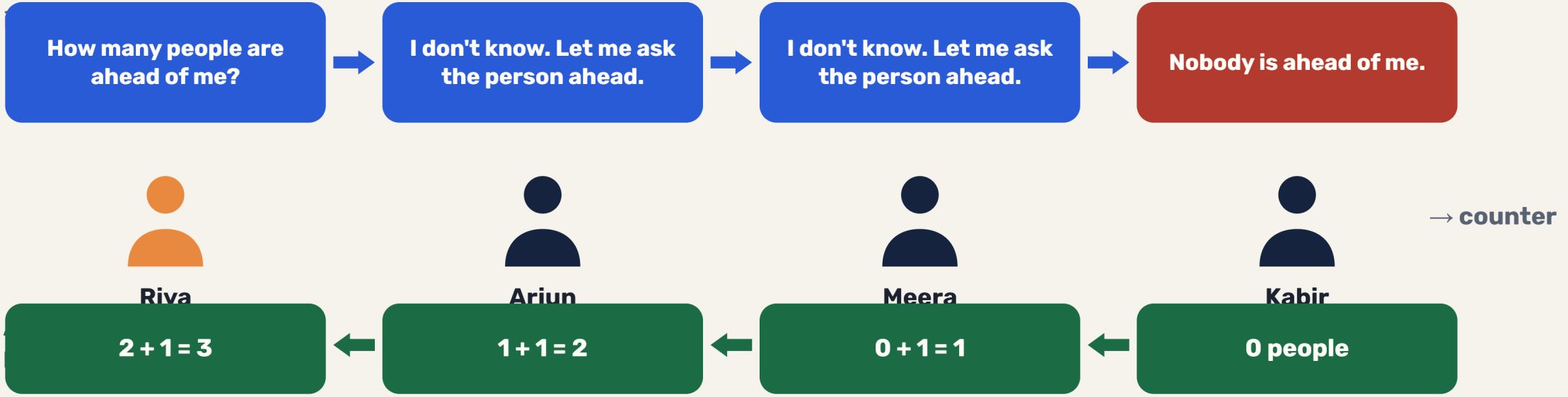
**Can she find out?**

Nobody can see the whole queue. Each person can only talk to the person **right ahead**. Let us help Riya.

BEFORE WE START • A REAL QUESTION

# How many people are ahead of Riya?

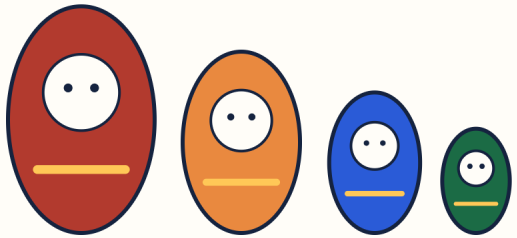
The canteen queue. Nobody can see past the person




**That is recursion** Each person asks the **same question**, one step smaller. The answers come back one by one.

TOPIC 1 OF 2 • RECURSIVE FUNCTIONS

# What is recursion? A smaller copy inside



Open a doll: a smaller doll is inside. Open that: a smaller one again.

**A recursive function calls itself**

```
solve(problem) =
  a little work + solve(smaller problem)
```

**Same job every time**

The queue

**ask the person ahead**

**The problem shrinks each call**

The dolls

**open the next doll**

**It stops at the smallest problem**

The code

**call the function again**

TOPIC 1 OF 2 • RECURSIVE FUNCTIONS

# Two parts of every recursive function



## 1 • The base case

The smallest problem. Answer it directly, with no more calls. It tells the function when to STOP.

In the queue: **the front person says 0**



## 2 • The recursive case

Call yourself with a smaller input, then use the answer to finish the job.

In the queue: **ask ahead, then add 1**

```
int ahead(int pos) {
    if (pos == 1)
        return 0;
    return 1 + ahead(pos - 1);
}
```

**Base case** lines 2 and 3

**Recursive case** line 4: a smaller pos

**No base case? The calls never stop, and the program crashes. We will see that soon.**

# The queue question, as code

```c
#include <stdio.h>

int ahead(int pos) {
    if (pos == 1)
        return 0;
    return 1 + ahead(pos - 1);
}

int main(void) {
    printf("%d\n", ahead(4));
    return 0;
}
```

**Memory: the call stack**

Coming back: answers return

`ahead(pos = 1)` — **returns 0**
base case? return 0

`ahead(pos = 2)` — **returns 1**
1 + ahead(1) = 1 + ?

`ahead(pos = 3)` — **returns 2**
1 + ahead(2) = 1 + ?

`ahead(pos = 4)` — **returns 3**
1 + ahead(3) = 1 + ?

`main()`
waiting for ahead(4) ...

**Output**

```
3
```

The line being run

A call waiting for its answer

An answer coming back

Each call waits in its own box until the call above it answers.

9 main prints 3: three people are ahead of Riya.

TOPIC 1 OF 2 • RECURSIVE FUNCTIONS • PREDICT

# What does it return?

**1** `ahead(1)`

**0: it is the base case**

**2** `ahead(2)`

**1: one person ahead**

**3** How many boxes are on the stack at the deepest point of `ahead(4)`?

**5: main plus four calls**

**4** What would `ahead(0)` do? (`pos` never equals 1)

**It never stops: stack overflow**

TOPIC 1 OF 2 • RECURSIVE FUNCTIONS

# Factorial: how many ways to line up 4 friends?




Four friends stand in a queue. **In how many different orders** can they stand?

**First place: 4 choices**

**Second place: 3 choices left**

**Third place: 2 choices left**

**Last place: 1 choice left**

$$4 \times 3 \times 2 \times 1 = 24$$

**The same thing, as a smaller problem**

$$4! = 4 \times 3!$$
$$3! = 3 \times 2!$$
$$2! = 2 \times 1!$$

**The smallest problem**

$$1! = 1$$

One friend can stand in 1 way. That is the base case.

# factorial(4) on the stack

```c
#include <stdio.h>

int factorial(int n) {
    if (n == 1)
        return 1;
    return n * factorial(n - 1);
}

int main(void) {
    printf("%d\n", factorial(4));
    return 0;
}
```

**Memory: the call stack**

Coming back: calls wake up and multiply

factorial(n = 1) — **returns 1**
base case: return 1

factorial(n = 2) — **returns 2**
2 × factorial(1) = 2 × ?

factorial(n = 3) — **returns 6**
3 × factorial(2) = 3 × ?

factorial(n = 4) — **returns 24**
4 × factorial(3) = 4 × ?

main()
waiting for factorial(4) ...

**Output**

24

The line being run

A call waiting for its answer

An answer coming back

Nothing is multiplied until the base case is reached.

9 main prints 24: four friends can line up in 24 ways.

TOPIC 1 OF 2 • RECURSIVE FUNCTIONS • PREDICT

# Trace it in your head

| | | |
|---|---|---|
| 1 | `factorial(3)` | 6: 3 × 2 × 1 |
| 2 | `factorial(5)` | 120: 5 × 24 |
| 3 | How many calls does `factorial(5)` make in total? | 5 calls: n = 5, 4, 3, 2, 1 |
| 4 | In `factorial(4)`, when is the first multiplication done? | After the base case returns 1 |

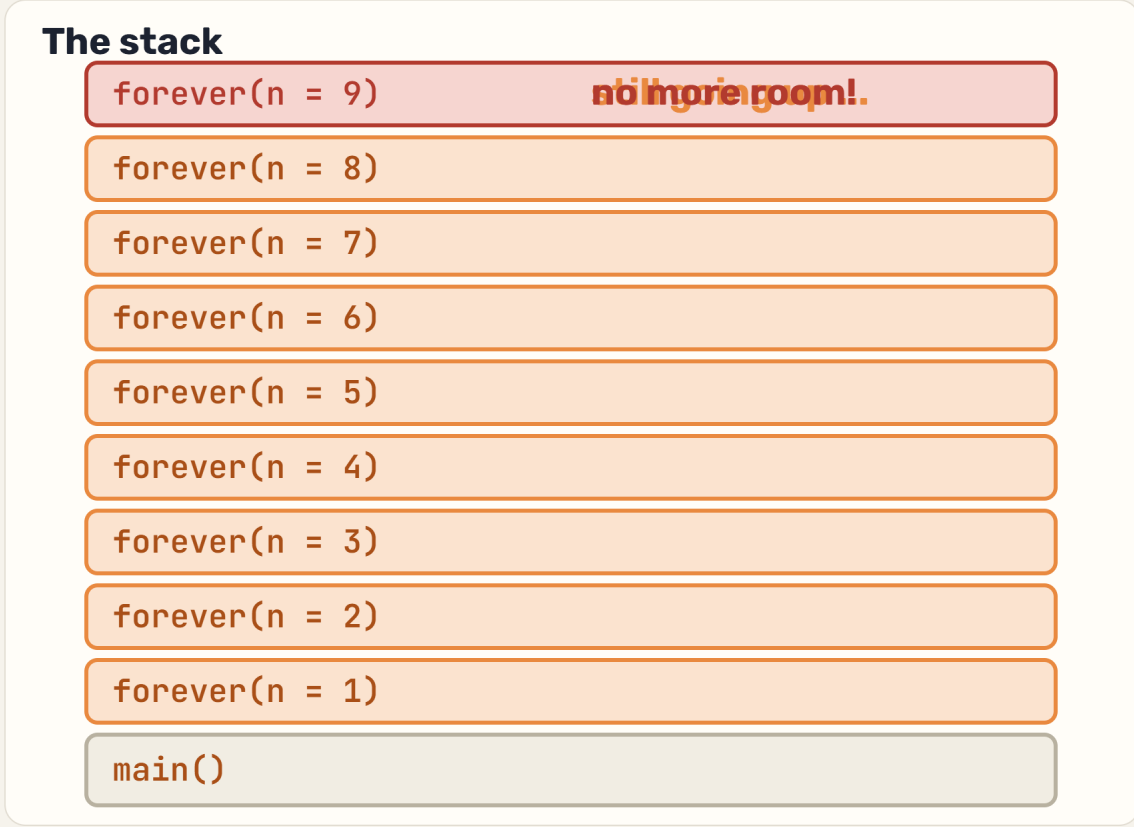
# No base case? The stack overflows

```
void forever(int n) {
    printf("%d ", n);
    forever(n + 1);
}
```

**Look** No **if**, no base case. Nothing says STOP.

**Stack overflow**

The stack runs out of room and the program crashes, usually with a Segmentation fault.



TOPIC 1 OF 2 • RECURSIVE FUNCTIONS

# Three ways recursion goes wrong

### 1 • No base case

```
int f(int n) {
    return n * f(n - 1);
}
```

✗ Stack overflow

**Fix:** Add: if (n == 1) return 1;

### 2 • It never gets smaller

```
int f(int n) {
    if (n == 1) return 1;
    return n * f(n);
}
```

✗ Stack overflow

**Fix:** Call f(n - 1), not f(n)

### 3 • The wrong base value

```
int f(int n) {
    if (n == 1) return 0;
    return n * f(n - 1);
}
```

✗ Always answers 0

**Fix:** The base case must return 1

YOUR TURN · PARSONS PROBLEM

# Put the lines in order to build sum_to

```
A   return n + sum_to(n - 1);
B   }
C   int sum_to(int n) {
D   if (n == 0)
E   return 0;
```

sum_to(n) adds 1 + 2 + ... + n. For example sum_to(3) = 6.

Write the order of letters. Hint: where must the base case go?

**C · D · E · A · B**

Header first, then the base case (D, E), then the recursive case (A). The stopping check must come BEFORE the recursive call, or the calls never end.

TOPIC 1 OF 2 • RECURSIVE FUNCTIONS • MISSING LINE CHALLENGE

# One line is missing: the program crashes

```c
/* One line is missing. Find it and write it. */
#include <stdio.h>

int sum_to(int n) {
    /* ??? missing line ??? */
    return n + sum_to(n - 1);
}

int main(void) {
    printf("%d\n", sum_to(3));
    return 0;
}
```

sum_to(3) should print **6**, but the program crashes with a stack overflow. Which line is missing?

**Correct output**

```
6
```

**Answer: if (n == 0) return 0;**

That is the base case. Without it, sum_to(0) calls sum_to(-1), then sum_to(-2), and so on, until the stack is full.

TRY IT • ONE USE CASE: THE LUNCH-BELL COUNTDOWN

# A simple recursion and its output

```c
#include <stdio.h>

void countdown(int n) {
    if (n == 0) {
        printf("Bell rings!\n");
        return;
    }
    printf("%d\n", n);
    countdown(n - 1);
}

int main(void) {
    countdown(3);
    return 0;
}
```

**Output**

```
3
2
1
Bell rings!
```

**Base case**

n == 0: ring the bell and stop.

**Recursive case**

Print n, then call countdown(n – 1): a smaller number.

**Read it** Each call prints its own n **first**, then asks the next call. So the numbers appear in the order 3, 2, 1.

TRY IT • ONE USE CASE: THE LUNCH-BELL COUNTDOWN • TRACE

# Countdown on the stack

```c
#include <stdio.h>

void countdown(int n) {
    if (n == 0) {
        printf("Bell rings!\n");
        return;
    }
    printf("%d\n", n);
    countdown(n - 1);
}

int main(void) {
    countdown(3);
    return 0;
}
```

**Memory: the call stack**

**Printing happens on the way down**

`countdown(n = 0)` **prints bell**
base case: rings the bell

`countdown(n = 1)` **prints 1**
printed 1, then asked countdown(0)

`countdown(n = 2)` **prints 2**
printed 2, then asked countdown(1)

`countdown(n = 3)` **prints 3**
printed 3, then asked countdown(2)

`main()`
waiting for countdown(3) ...

**Output**

```
3
2
1
Bell rings!
```

The line being run

A call waiting for its answer

An answer coming back

Each call prints, then goes deeper.

9 Now every call returns. Nothing is left to print.

YOUR TURN • MODIFY THE PROGRAM IN ONLINEGDB

# Change one line, predict the output

**Open countdown.c in OnlineGDB. Change one line so that the numbers print as 1 2 3 instead of 3 2 1.**

First write down: what will the output be? Then run it.

**Start from this program**

Run it first. You should see 3, 2, 1, Bell rings!

onlinegdb.com/xlqcZ3SEH

The **bell rings first**: the base case runs before any number is printed. The numbers print on the way back.

**Hint: print n only after the recursive call has finished.**

```
3  void countdown(int n) {
4      if (n == 0) {
5          printf("Bell rings!\n");
6          return;
7      }
8      countdown(n - 1);
9      printf("%d\n", n);
10 }
```

**New output**

```
Bell rings!
1
2
3
```

TRY IT • ONE USE CASE: THE LUNCH-BELL COUNTDOWN • TRACE

# Why the bell rings first

```
1  #include <stdio.h>
2
3  void countdown(int n) {
4      if (n == 0) {
5          printf("Bell rings!\n");
6          return;
7      }
8      countdown(n - 1);
9      printf("%d\n", n);
10 }
11
12 int main(void) {
13     countdown(3);
14     return 0;
15 }
```

**Memory: the call stack**

Going down: the numbers are printed

Coming back: the numbers are printed

`countdown(n = 0)` — **prints bell**
base case: rings the bell

`countdown(n = 1)` — **prints 1**
printed after countdown(0), then prints 1

`countdown(n = 2)` — **prints 2**
printed after countdown(1), then prints 2

`countdown(n = 3)` — **prints 3**
printed after countdown(2), then prints 3

`main()`
waiting for countdown(3) ...

**Output**

```
Bell rings!
1
2
3
```

The line being run

A call waiting for its answer

An answer coming back

Printing now waits until the call below has returned.

9 main continues. The output is: Bell rings!, 1, 2, 3.

YOUR TURN • LEVEL 2

# Now make the bell ring last

**Change the program so that the output is:**

```
1
2
3
Bell rings!
```

**Hint: the bell is not part of the counting. Which function can ring it after countdown has finished?**

The base case now does **nothing**, and **main** rings the bell after countdown(3) is done.

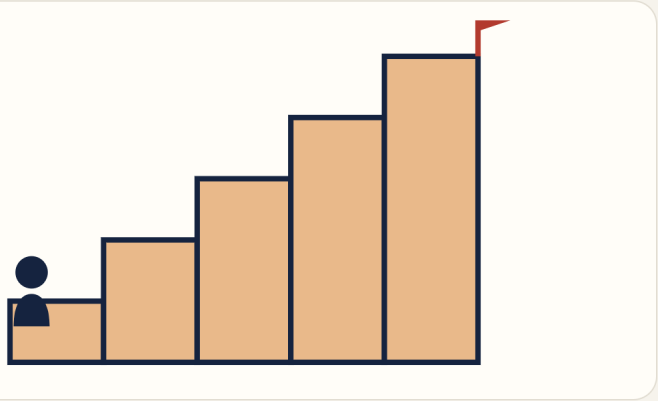
```
3  void countdown(int n) {
4      if (n == 0)
5          return;
6      countdown(n - 1);
7      printf("%d\n", n);
8  }
9
10 int main(void) {
11     countdown(3);
12     printf("Bell rings!\n");
13     return 0;
14 }
```

**Output**

```
1
2
3
Bell rings!
```

TOPIC 1 OF 2 • RECURSIVE FUNCTIONS

# Climbing the stairs: how many ways?



**The library has 5 steps. You may climb 1 step or 2 steps at a time. How many different ways reach the top?**

| | |
|---|---|
| `ways(1) = 1` | 1 |
| `ways(2) = 2` | 1+1, 2 |
| `ways(3) = 3` | 1+1+1, 1+2, 2+1 |

Your **last move** was a 1-step from step n – 1, or a 2-step from step n – 2. So:

```
ways(n) = ways(n - 1) + ways(n - 2)
```

```
int ways(int steps) {
    if (steps ⩽ 2)
        return steps;
    return ways(steps - 1) + ways(steps - 2);
}
```

**New idea** One call can make **two** recursive calls. The base case is steps <= 2.

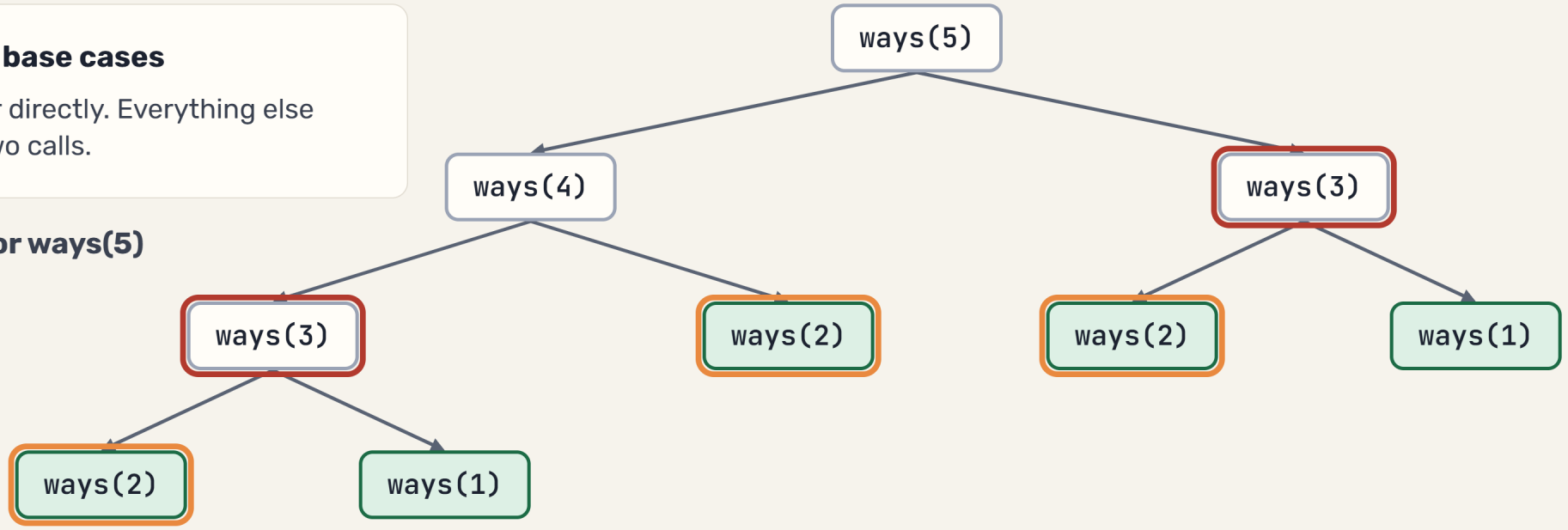
TOPIC 1 OF 2 • RECURSIVE FUNCTIONS

# The call tree of ways(5)

**Green: the base cases**

They answer directly. Everything else splits into two calls.

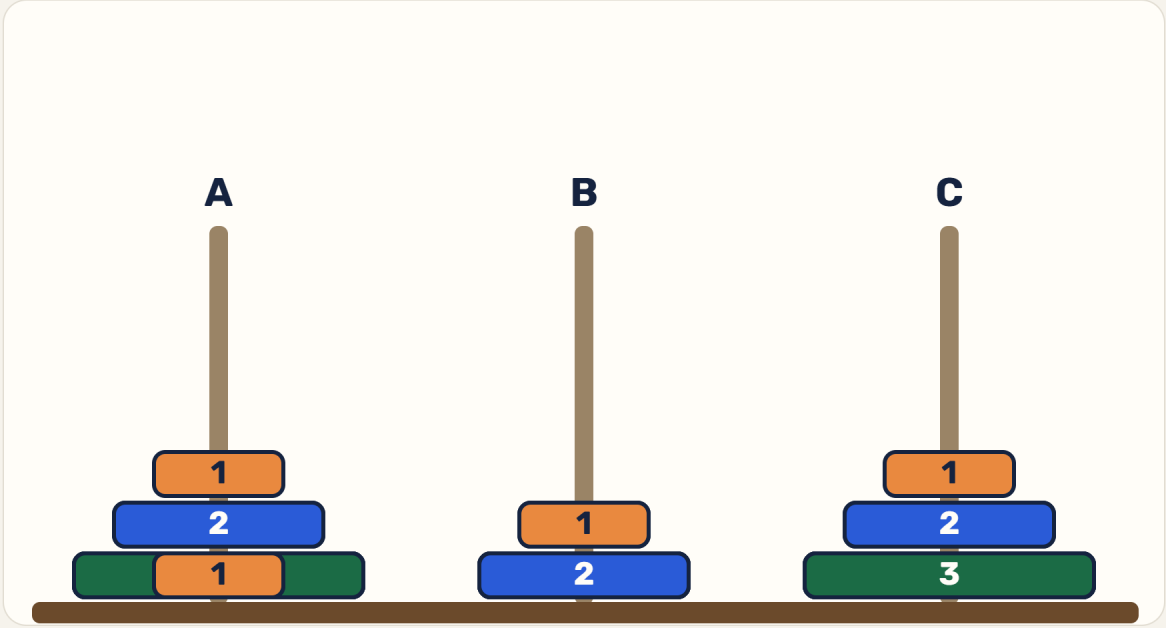
**9 calls just for ways(5)**




**Repeated work** **ways(3)** is worked out twice and **ways(2)** three times. The same answers are calculated again and again.

TOPIC 1 OF 2 • RECURSIVE FUNCTIONS

# Tower of Hanoi: recursion makes it easy




1 Move the whole tower from A to C.
2 One disc at a time.
3 Never a big disc on a small one.

```
void hanoi(int n, char a, char c, char b) {
    if (n == 0)
        return;
    hanoi(n - 1, a, b, c);
    printf("Disc %d: %c to %c\n", n, a, c);
    hanoi(n - 1, b, c, a);
}
```

**Moves so far: 7**

```
1  Disc 1: A to C
2  Disc 2: A to B
3  Disc 1: C to B
4  Disc 3: A to C
5  Disc 1: B to A
6  Disc 2: B to C
7  Disc 1: A to C
```

**Solved in 7 moves**

TOPIC 1 OF 2 • RECURSIVE FUNCTIONS

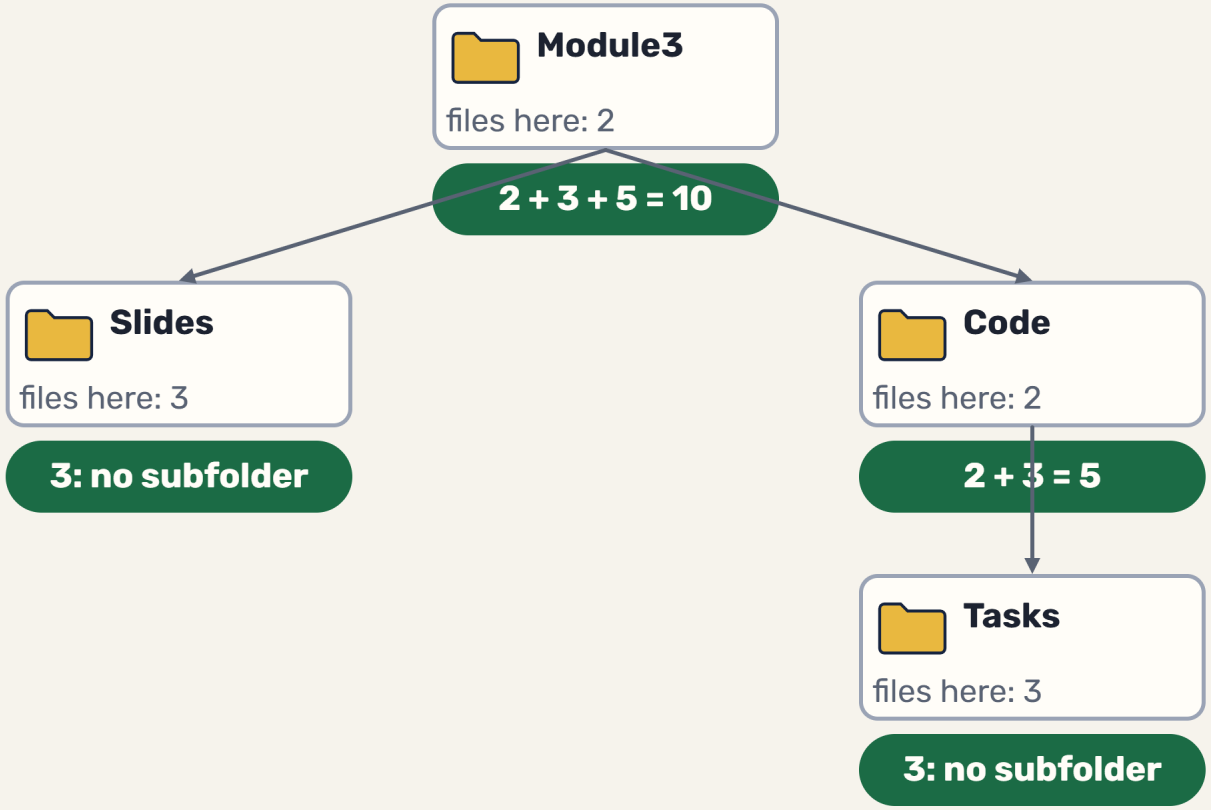
# How many moves for 64 discs?

| | | |
|---|---|---|
| **1 disc** | `1 move` | $2^1 - 1$ |
| **2 discs** | `3 moves` | $2^2 - 1$ |
| **3 discs** | `7 moves` | $2^3 - 1$ |
| **10 discs** | `1,023 moves` | $2^{10} - 1$ |
| **64 discs** | `18,446,744,073,709,551,615 moves` | $2^{64} - 1$ |

**You have seen this number** It is the total number of rice grains on the whole chessboard from Module 2, Class 3. One move per second would take about 585 billion years.

TOPIC 1 OF 2 • RECURSIVE FUNCTIONS

# Folders inside folders: recursion at work




**The idea (pseudo-code)**

```
count(folder):
    total = files here
    for each subfolder:
        total += count(sub)
    return total
```

**Base case** A folder with no subfolders: just count its files.

**Natural recursion** Each subfolder is a smaller copy of the same problem. A loop would struggle here.

TOPIC 2 OF 2 · RECURSION VS ITERATION

# Same answer, two ways: a loop or a recursion

## Iteration: a loop

```
int factorial(int n) {
    int result = 1, i;
    for (i = 2; i ⩽ n; i++)
        result = result * i;
    return result;
}
```

**Memory: one box, reused**

```
result = 24
```

## Recursion: it calls itself

```
int factorial(int n) {
    if (n == 1)
        return 1;
    return n * factorial(n - 1);
}
```

```
factorial(n = 1)
factorial(n = 2)
factorial(n = 3)
factorial(n = 4)
```

**Both print 24** The loop uses **one** box. The recursion uses **four**, and n boxes for n.

TOPIC 2 OF 2 · RECURSION VS ITERATION

# The cost of repeating work

The same stairs question, ways(n), solved by recursion and by a loop. Counted by running the program.

| n | Recursive calls | Loop rounds | How much more work |
|---|---|---|---|
| 10 | 109 | 8 | 14 times |
| 20 | 13,529 | 18 | 750 times |
| 30 | 1,664,079 | 28 | 59,000 times |
| 40 | 204,668,309 | 38 | 5 million times |

**Why ?** The call tree doubles as n grows, and the same small problems are solved over and over. The loop solves each one once.

TOPIC 2 OF 2 • RECURSION VS ITERATION

# Which one should I use?

## Use a loop when

- You just count, add or print 1 to n
- Speed and memory matter
- A simple loop says it clearly

## Use recursion when

- The problem holds smaller copies of itself: folders, Hanoi
- The recursive version is much clearer to read
- Later: trees and divide-and-conquer ideas

**The rule** Anything a recursion can do, a loop can do too, but sometimes the loop is long and ugly. Choose the simplest tool that is **clear**.

TOPIC 2 OF 2 • RECURSION VS ITERATION • PREDICT

# Loop or recursion?

**1** Which uses more memory for n = 1000: the loop or the recursion?

**Recursion: about 1000 boxes, the loop has 1**

**2** ways(40) by recursion or by a loop: which finishes at once?

**The loop: the recursion makes about 200 million calls**

**3** Count all the files in folders inside folders. Easier with:

**Recursion: each subfolder repeats the problem**

**4** True or false: anything recursion can do, a loop can do too.

**True, though the loop can be harder to write**

THE PROGRAM, PART 1 OF 3

# Five questions, five answers

3–7 **prototypes: announce all five functions**

9–16 **main asks five questions and prints each answer**

```c
#include <stdio.h>

int ahead(int pos);
int factorial(int n);
int digit_sum(int n);
int ways(int steps);
int hanoi_moves(int n);

int main(void) {
    printf("People ahead of position 4: %d\n", ahead(4));
    printf("Ways to line up 4 friends:
%d\n", factorial(4));
    printf("Digit sum of 2026: %d\n", digit_sum(2026));
    printf("Ways to climb 5 steps: %d\n", ways(5));
    printf("Hanoi moves for 3 discs: %d\n",
hanoi_moves(3));
    return 0;
}
```

THE PROGRAM, PART 2 OF 3

# Three recursions you know

18–22 **ahead: base case pos == 1, then ask ahead and add 1**

24–28 **factorial: base case n == 1, multiply on the way back**

30–34 **digit_sum: base case n == 0; n % 10 is the last digit, n / 10 drops it**

```
18  int ahead(int pos) {
19      if (pos == 1)
20          return 0;
21      return 1 + ahead(pos - 1);
22  }
23
24  int factorial(int n) {
25      if (n == 1)
26          return 1;
27      return n * factorial(n - 1);
28  }
29
30  int digit_sum(int n) {
31      if (n == 0)
32          return 0;
33      return n % 10 + digit_sum(n / 10);
34  }
```

THE PROGRAM, PART 3 OF 3

# Two with a twist

36–40 **ways: TWO recursive calls, base case steps <= 2**

42–46 **hanoi_moves: 2 × moves(n − 1) + 1, base case n == 0**

```
36  int ways(int steps) {
37      if (steps ⩽ 2)
38          return steps;
39      return ways(steps - 1) + ways(steps - 2);
40  }
41
42  int hanoi_moves(int n) {
43      if (n == 0)
44          return 0;
45      return 2 * hanoi_moves(n - 1) + 1;
46  }
```

RUN IT YOURSELF

# Run the recursion lab, then check your guesses

`recursion_lab.c`

1. Open the link or scan the QR code
2. This program needs no input
3. Press **Run**
4. Check the last line: **Hanoi moves for 3 discs: 7**



onlinegdb.com/i-mooUt_Zq

PREDICT, THEN RUN TO CHECK

# What will each line print?

1 `digit_sum(2026)`: what is the base case?

**n == 0: no digits left**

2 Which function makes TWO recursive calls?

**ways**

3 `hanoi_moves(4)`?

**15: 2 × 7 + 1**

4 What does `digit_sum(2026)` print, and which digit does it add first?

**10: it adds the last digit, 6, first**

YOUR TURN • TASK 1 OF 3

# Add up 1 to n



Write sum_to(n) that adds 1 + 2 + ... + n, with recursion and no loop.

## Plan the recursion

| | | |
|---|---|---|
| 1 | **Name** | `sum_to` |
| 2 | **Inputs** | `int n` |
| 3 | **Gives back** | `int: the sum 1 to n` |
| 4 | **Base case** | `n == 0 returns 0` |
| 5 | **Recursive** | `n + sum_to(n - 1)` |
| 6 | **Test** | `sum_to(4) gives 10` |

## Sample runs

```
sum_to(4) = 10
sum_to(10) = 55
sum_to(0) = 0
```

## Check yourself

- ✓ What is the smallest problem?
- ✓ Does each call get smaller?
- ✓ Test 0 and 1

**Now:** Write the base case first, then the recursive case. Test every sample.

SOLUTION · TASK 1: SUM_TO

# sum_to on the stack

```
1  #include <stdio.h>
2
3  int sum_to(int n) {
4      if (n == 0)
5          return 0;
6      return n + sum_to(n - 1);
7  }
8
9  int main(void) {
10     printf("%d\n", sum_to(3));
11     return 0;
12 }
```

**Memory: the call stack**

Coming back: answers return

`sum_to(n = 0)` — **returns 0**
`base case: return 0`

`sum_to(n = 1)` — **returns 1**
`1 + sum_to(0) = 1 + ?`

`sum_to(n = 2)` — **returns 3**
`2 + sum_to(1) = 2 + ?`

`sum_to(n = 3)` — **returns 6**
`3 + sum_to(2) = 3 + ?`

`main()`
`waiting for sum_to(3) ...`

**Output**

```
6
```

The line being run

A call waiting for its answer

An answer coming back

Check your own answer against the boxes.

9 main prints 6.

YOUR TURN • TASK 2 OF 3

# Power without pow




Write power(base, exp) that works out base to the power exp, using recursion.

## Plan the recursion

| | | |
|---|---|---|
| 1 | **Name** | `power` |
| 2 | **Inputs** | `int base, int exp` |
| 3 | **Gives back** | `int: base to the power exp` |
| 4 | **Base case** | `exp == 0 returns 1` |
| 5 | **Recursive** | `base * power(base, exp - 1)` |
| 6 | **Test** | `power(2, 10) gives 1024` |

## Sample runs

```
power(2, 3) = 8
power(2, 10) = 1024
power(5, 0) = 1
```

## Check yourself

- ✓ Anything to the power 0 is 1
- ✓ Only exp changes, not base
- ✓ Which argument gets smaller?

**Now:** Write the base case first, then the recursive case. Test every sample.

SOLUTION · TASK 2: POWER

# power on the stack

```
1  #include <stdio.h>
2
3  int power(int base, int exp) {
4      if (exp == 0)
5          return 1;
6      return base * power(base, exp - 1);
7  }
8
9  int main(void) {
10     printf("%d\n", power(2, 3));
11     return 0;
12 }
```

**Memory: the call stack**

Coming back: answers return

power(base = 2, exp = 0) — returns 1
base case? return 1

power(base = 2, exp = 1) — returns 2
2 × power(2, 0) = 2 × ?

power(base = 2, exp = 2) — returns 4
2 × power(2, 1) = 2 × ?

power(base = 2, exp = 3) — returns 8
2 × power(2, 2) = 2 × ?

main()
waiting for power(2, 3) ...

**Output**

```
8
```

The line being run

A call waiting for its answer

An answer coming back

Check your own answer against the boxes.

9 main prints 8.

YOUR TURN • TASK 3 OF 3

# Simplify a fraction




The gcd of two numbers is the biggest number that divides both. Divide top and bottom by it to simplify 48/18. Euclid's idea: gcd(a, b) = gcd(b, a % b).

## Plan the recursion

| | | |
|---|---|---|
| 1 | **Name** | `gcd` |
| 2 | **Inputs** | `int a, int b` |
| 3 | **Gives back** | `int: the biggest common divisor` |
| 4 | **Base case** | `b == 0 returns a` |
| 5 | **Recursive** | `gcd(b, a % b)` |
| 6 | **Test** | `gcd(48, 18) gives 6` |

## Sample runs

```
gcd(48, 18) = 6
gcd(17, 5) = 1
gcd(10, 0) = 10
```

## Check yourself

- ✓ The remainder is always smaller
- ✓ When b becomes 0, a is the answer
- ✓ 48/18 becomes 8/3

**Now:** Write the base case first, then the recursive case. Test every sample.

SOLUTION · TASK 3: GCD

# gcd on the stack

```c
#include <stdio.h>

int gcd(int a, int b) {
    if (b == 0)
        return a;
    return gcd(b, a % b);
}

int main(void) {
    printf("%d\n", gcd(48, 18));
    return 0;
}
```

**Memory: the call stack**

Coming back: answers return

gcd(a = 6, b = 0) — returns 6
base case: return a = 6

gcd(a = 12, b = 6) — returns 6
same answer

gcd(a = 18, b = 12) — returns 6
same answer

gcd(a = 48, b = 18) — returns 6
same answer

main()
waiting for gcd(48, 18) ...

**Output**

```
6
```

The line being run

A call waiting for its answer

An answer coming back

Check your own answer against the boxes.

9 main prints 6: the fraction 48/18 simplifies to 8/3.

TODAY IN ONE SLIDE

# The function that calls itself

1

### Two parts

A base case that stops, and a recursive case that calls itself with a smaller problem.

2

### The stack

Calls pile up on the way down. Answers come back, and boxes pop, on the way up.

3

### Loop or recursion

A loop is cheap and simple. Recursion is clear when the problem holds smaller copies of itself.

Before writing any recursion, ask: what is the base case, does every call get smaller, and does the base case return the right value?

MODULE 3 • COMPLETE

# From one big program to small, named blocks



**Functions**

a named block of work



**Copy or key**

value and reference



**The stack**

a pile of boxes



**Scope and storage**

who sees it, how long it lives



**Recursion**

a function calls itself

**You can now** build programs from small, reusable, named blocks, and understand exactly what the computer does with each call.

COMING NEXT • MODULE 4

# Riya's menu has 50 items. Do we need 50 variables?

Module 4: arrays and strings.

Before next class: finish one task, and try recursion_lab.c with your own numbers.